Text passed through the analyzer is converted and stored as a keyword. For example, if we pass "Elasticsearch in Action" through the `keyword` analyzer, the whole text string is stored as is, unlike the way earlier analyzers split the text into tokens.

Listing 7.14 Using the `keyword` analyzer

```
POST _analyze
{
  "text":"Elasticsearch in Action",
  "analyzer": "keyword"
}
```

The output of this script is shown here:

```
"tokens" : [{
  "token" : "Elasticsearch in Action",
  "start_offset" : 0,
  "end_offset" : 23,
  "type" : "word",
  "position" : 0
}]
```

As you can see, only one token was produced as a result of processing the text via the `keyword` analyzer. There's also no lowercasing. However, there is a change in how we search if we use the `keyword` analyzer to process text. Searching for a single word will not match the text string—we must provide an exact match. In our example, we must provide the exact group of words as in the original sentence: "Elasticsearch in Action".

7.3.5 The fingerprint analyzer

The `fingerprint` analyzer removes duplicate words and extended characters and sorts the words alphabetically to create a single token. It consists of a `standard` tokenizer and four token filters: `fingerprint`, `lowercase`, `stop`, and ASCII folding (`asciifolding`) token filters (see figure 7.10).

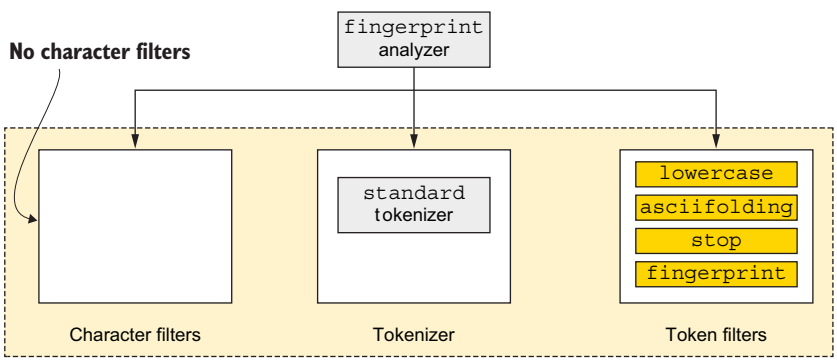


Figure 7.10 Anatomy of the `fingerprint` analyzer